

1. Performance Analysis:

Buffer Size	total runtime
1 byte	11m 14.931s
64 bytes	14.479s
512 bytes	4.750s
1024 bytes	3.760s
8192 bytes	2.817s
65536 bytes	2.727s

- We see that the larger the buffer, the faster the Copy program runs.

This behavior is expected, because in every iteration we have 2 syscalls - a `read()` and a `write`. With a smaller buffer we need more iterations to copy the entire file, thus more syscalls and a longer runtime due to the exception handling overhead.

With a 100MB file to copy, the 65536B buffer run only does 1600 iterations (3200 syscalls), while the 1B buffer run does a ridiculous amount of $100 \cdot 2^{20}$ iterations ($100 \cdot 2^{21}$ syscalls).

2. O_EXCL option:

* Based on the info found in the manual pages: if this flag is present alongside `O_CREAT` - `open()` doesn't allow for the file address to exist and if it does an `EEXIST` error is thrown and the `open()` operation is failed. If `O_EXCL` is used without `O_CREAT` - we can get unexpected behavior.

* Based on ChatGPT: "`O_EXCL` means 'exclusive creation' and it is almost always used together with `O_CREAT`."

For example, passing "file.txt" to `open()` with the flag combination `O_CREAT | O_EXCL | O_WRONLY` will create the file if it doesn't exist, and if it does it will fail - returns -1 and sets `errno` to `EEXIST`. This makes files creation atomic, which is important for avoiding race conditions." Later it gives some code examples.

* This shows how ChatGPT is more of a 'generalist' - it goes less into detail than the official documentation and gives us real code examples straight away, but failed on reporting the possibility for an undefined behavior - which is a big miss.

* The shell command "man open" opens the documentation page for the shell command "open" (or "xdg-open"), and in that way it differs from "man 2 open" which opens the doc. page for the system call `open()`.

3. AI Experiment:

I chose ChatGPT for this assignment. My initial prompt was:

I need you to create a C program based on the following requirements:

Write/Code a program that copies a file's content to another file. Name your source file copy.c. The program should print an error message and exit if there aren't enough parameters to the program, if the input file doesn't exist, or if the output file exists. In addition, your program must handle errors from system calls as described in the Error Handling subsection below. The program should use the system calls open(), close(), read() and write(). The copying should be done using a buffer of size n bytes, meaning the program reads n bytes in one read, writes them, and continues in a loop. - error handling: for logic/input error you should handle like this:

```
fprintf(stderr, "not enough arguments");  
exit(1);
```

and for system calls failures like this:

```
if (fd == -1) {  
    perror("open");  
    exit(1);  
}
```

in any case - the program should clean up any resources (allocated memory, open files) before exiting.

the program receives 3 input arguments: input file name, output file name, and n.

Most requirement it got in the first try: Compiled with no errors/warning, Copied the files correctly with the appropriate flags

For both the input file open() and the output file open(), and it passed all tests except for a minor mistake in error handling:

printed "not enough arguments" for launches like `./chat testfile output -3` or `./chat testfile output x`.

Follow up prompts:

I get the "not enough arguments" error in the following cases:

`./chat testfile output s`

`./chat testfile output1 -3`

`./chat testfile output1 4.5`

while I should get a different error which reflects the situation better.

also you should add a line break after the "not enough arguments" error message you print to improve readability

- after which a single fix from the model was enough to achieve all the requirements.